SASTRA DEEMED TO BE UNIVERSITY
SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CAPSTONE PROJECT REPORT

AUTOSAR AI – HIGH-LEVEL DESIGN DOCUMENT INTELLIGENCE PLATFORM

Submitted in partial fulfillment of the requirements for the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING (AIML)

Submitted by:
HAMDHAN AHMED B
Register Number: 127003081

OCTOBER 2026
THANJAVUR, TAMIL NADU, INDIA

CERTIFICATE

This is to certify that the project report entitled “AUTOSAR AI – High-Level Design Document Intelligence Platform” is a bonafide record of work done by Hamdhan Ahmed B (Register Number: 127003081) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering (AIML) from SASTRA Deemed University, Thanjavur, during the academic year 2025–2026.

Project Guide: _____ [Faculty Guide Name] Assistant Professor, School of Computing	Head of Department: _____ [HOD Name] Head, Department of CSE, School of Computing
Internal Examiner: _____	External Examiner: _____

TABLE OF CONTENTS

Chapter Title	Page No.
1. Introduction & Domain Overview	1
2. Problem Statement & Objectives	3
3. Literature Survey & Existing Approaches	5
4. System Architecture & Requirements	7
5. Methodology & Implementation Details	10
6. User Interface & Module Descriptions	14
7. Testing Methodology & Empirical Results	18
8. Security, Privacy & Safety Considerations	22
9. Limitations & Future Scope	24
10. Conclusion & References	26

CHAPTER 1: INTRODUCTION & DOMAIN OVERVIEW

The contemporary automotive industry is experiencing a profound transition from hardware-centric mechanical systems to Software-Defined Vehicles (SDVs). Modern connected, automated, and electrified vehicles integrate upwards of one hundred Electronic Control Units (ECUs) executing over one hundred million lines of code. To manage this extraordinary architectural complexity, major automotive manufacturers (OEMs) and Tier-1 suppliers established the AUTOSAR (AUTomotive Open System ARchitecture) consortium. AUTOSAR standardizes the software architecture of automotive ECUs, enforcing a strict separation between application-layer software components and the underlying microcontroller hardware through a standardized Runtime Environment (RTE) and Basic Software (BSW) stack.

Within the automotive V-model development lifecycle, the architectural blueprint for any subsystem is formalized in a High-Level Design (HLD) document. An HLD specifies Software Components (SWCs), their internal execution tasks (Runnables), inter-component communication contracts (Interfaces), interaction points (Ports), and individual carried signals along with their physical units and data types. These documents frequently span hundreds of pages, evolving across multiple revision cycles as vehicle requirements change. Reviewing and verifying these documents is traditionally a manual, labor-intensive process that consumes thousands of senior engineering hours and remains prone to overlooked interface mismatches, unintended circular dependencies, and lost traceability.

AUTOSAR AI was developed to address this operational bottleneck by introducing an on-premises, high-assurance document intelligence platform. Designed specifically for automotive systems engineers, the platform automates HLD ingestion, structural entity extraction, architectural rule checking, revision diffing, and grounded conversational question-answering backed by verifiable page citations and empirical claim verification scores.

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

Automotive software engineering environments present three critical constraints that preclude the direct adoption of generic cloud-based AI tools:
1. Strict Intellectual Property & Privacy Regulations: Under automotive cybersecurity standards (ISO/SAE 21434) and corporate non-disclosure agreements, proprietary vehicle software architectures cannot be transmitted to external cloud LLM providers.
2. Intolerance for AI Hallucinations: In safety-critical vehicle engineering governed by ISO 26262, ungrounded answers or fabricated component names can compromise functional safety audits.
3. Lack of Exact Traceability: Generic RAG pipelines frequently return broad text chunks without pin-point citation traceability down to physical PDF page numbers and claim-level verification.

To resolve these challenges, AUTOSAR AI achieves the following primary engineering objectives:
• High-Speed Page-Preserving Ingestion: Ingest complex HLD PDFs while strictly recording physical page bounds and section headers.
• Hybrid Dense-plus-BM25 Retrieval: Combine 384-dimensional dense semantic vectors with BM25 keyword matching via Reciprocal Rank Fusion to accurately retrieve both conceptual passages and

exact automotive identifiers.
• On-Premises Generative Inference: Execute local inference using the quantized llama3.2:3b model via Ollama with strict evidence-bounded prompt engineering.
• Claim-Level Verification & Grounding Score: Automatically decompose generated answers into verifiable claims and calculate a quantitative Grounding Score (0.0 to 1.0).
• Architectural Entity Extraction & Rule Checking: Parse SWCs, Runnables, Ports, and Signals into a relational database and detect circular dependencies and insecure IPC bindings.

CHAPTER 3: LITERATURE SURVEY & EXISTING APPROACHES

Traditional approaches to automotive document management rely on relational document repositories or ALM tools (such as PTC Integrity, Siemens Polarion, or IBM DOORS). While these platforms excel at requirement tracking, they require engineers to manually author and maintain every traceability link. When requirements change, traceability links frequently decay, leading to stale architectural documentation.

Recent advances in Natural Language Processing have popularized Retrieval-Augmented Generation (RAG) frameworks like LangChain and LlamaIndex. However, existing commercial implementations are predominantly designed for cloud environments and general-purpose customer support. They treat technical specifications as generic prose, struggling to distinguish subtle automotive identifiers (e.g., confusing `CanIf_Transmit` with `CanTp_Transmit`) and lacking claim-level verification metrics. AUTOSAR AI bridges this gap by coupling hybrid retrieval with domain entity regex extraction, local model execution, and spreadsheet injection defenses.

CHAPTER 4: SYSTEM ARCHITECTURE & REQUIREMENTS

The AUTOSAR AI platform is designed around a modular, decoupled architecture comprising eight distinct layers:
1. Presentation Layer: An interactive 9-page Streamlit web user interface (port 8501) providing dashboard metrics, document management, architecture visualization, AI assistance, revision diffing, findings review, and system diagnostics.
2. API Gateway & Routing Layer: A high-performance FastAPI asynchronous REST service (port 8000) structured across 10 dedicated routers.
3. Ingestion & Extraction Layer: C-speed PDF parsing via PyMuPDF (fitz) that segments text into 1000-character chunks with 150-character overlap.
4. Embedding & Vector Storage: SentenceTransformers (all-MiniLM-L6-v2) producing 384-dimensional dense vectors stored in persistent ChromaDB collections.
5. Hybrid Retrieval & Fusion: A two-stage pipeline combining cosine similarity search with an in-memory BM25 index using Reciprocal Rank Fusion (k=60).
6. Local LLM Inference: On-premises Ollama runtime (port 11434) executing llama3.2:3b with an evidence-bounded automotive system prompt.
7. Relational Persistence: SQLite 3 database managed via SQLAlchemy 2.0 ORM with 14 relational tables.

8. Security & Reporting: Automated CSV findings report generation with formula injection sanitization.

CHAPTER 5: METHODOLOGY & IMPLEMENTATION DETAILS

The core methodology follows a four-stage pipeline: (1) Ingest & Extract; (2) Index & Partition; (3) Retrieve & Generate; and (4) Decompose & Verify.

In Stage 1, PyMuPDF parses uploaded PDFs page by page. Heuristic regular expressions scan text chunks to extract Software Components, Runnables, Interfaces, Ports (PPorts, RPorts), and Signals. Entities are stored in the SQLite `architecture_entities` table with source page numbers.

In Stage 2, text chunks are converted into 384-dimensional dense vectors and saved to ChromaDB collections partitioned by project ID (`project_{project_id}`), guaranteeing complete multi-project data isolation.

In Stage 3, when a user queries the system, the hybrid retriever executes dense vector search and sparse BM25 scoring in parallel, merging ranks via Reciprocal Rank Fusion ($k=60$). The Top-5 chunks form the evidence context for local Ollama inference.

In Stage 4, `grounding_service.py` splits the LLM response into atomic sentences, verifies each against citation text, and calculates the quantitative Grounding Score ($GS = N_{grounded} / N_{total}$). Responses are classified into High, Medium, or Needs Review confidence tiers.

CHAPTER 6: USER INTERFACE & IMPLEMENTED MODULES

The platform delivers nine comprehensive, functional modules:
1. Dashboard (`1_Dashboard.py`): Overview of active projects, indexed documents, and subsystem health indicators.
2. Documents (`2_Documents.py`): PDF upload, version tagging, and governance lifecycle states (Draft -> In Review -> Approved).
3. Architecture Explorer (`3_Architecture_Explorer.py`): Extracted entity registers and interactive Mermaid dependency graphs.
4. AI Assistant (`4_AI_Assistant.py`): Grounded technical QA interface with source citation cards and Grounding Score tags.
5. Comparisons (`5_Comparisons.py`): Cross-revision diffing with component rename detection and signal data-type changes.
6. Findings (`6_Findings.py`): Automated architectural rule violations (circular dependencies, insecure IPC) and review workflows.
7. Reports (`7_Reports.py`): On-demand CSV export generation with audit logging.
8. Settings & Diagnostics (`8_Settings.py`): Parameter configuration and live 6-subsystem health probes.
9. Evaluation (`9_Evaluation.py`): Standardized 35-query benchmark execution and performance visualization.

CHAPTER 7: TESTING METHODOLOGY & EMPIRICAL RESULTS

To ensure high engineering quality, the entire application was subjected to rigorous automated testing using Pytest 9.1.1 on Python 3.11.9 (Windows 11). The automated test suite consists of 28 unit and integration tests across 12 test modules. All 28 tests pass with a 100% pass rate in 27.35 seconds.

Subsystem Tested	Test Module	Result
Security & Reporting	test_comparison_and_reports.py	3/3 PASSED (Formula sanitization, CSV download)
Revision Comparison	test_comparisons_and_renames.py	1/1 PASSED (Rename detection, data type diffs)
Project & Doc APIs	test_docs.py	3/3 PASSED (CRUD, input validation)
Ingestion & Vector DB	test_document_pipeline.py	2/2 PASSED (PyMuPDF chunking, ChromaDB cascade)
Grounding & Traceability	test_grounding_and_traceability.py	4/4 PASSED (Claim segmentation, Grounding Score)
Hybrid Retrieval & Eval	test_hybrid_retrieval_and_eval.py	3/3 PASSED (BM25 scoring, benchmark run)

Furthermore, empirical evaluation across our curated 35-query benchmark dataset demonstrated that the hybrid retriever successfully retrieves exact technical identifiers, while achieving a 100% abstention rate on out-of-scope adversarial questions.

CHAPTER 8: SECURITY, PRIVACY & SAFETY CONSIDERATIONS

Security and privacy were primary design criteria for AUTOSAR AI:
• Complete On-Premises Isolation: 100% of data processing occurs on localhost without external API calls.
• CSV Formula Injection Defense: Exported text cells starting with formula trigger characters (=, +, -, @) are sanitized by prepending a single quote (').
• Indirect Prompt Injection Boundaries: The system prompt explicitly instructs the LLM to treat document text as untrusted data.
• Multi-Tenant Project Isolation: Relational tables enforce project foreign keys, and ChromaDB

maintains dedicated collections per project.
• Zero Hardcoded Secrets: All environment configuration uses sanitized templates (.env.example) with placeholder values.

CHAPTER 9: LIMITATIONS & FUTURE SCOPE

In adherence to academic integrity, the following limitations of the current implementation are acknowledged:
1. Ingestion relies on text-searchable PDFs; scanned raster bitmaps require external OCR preprocessing.
2. Entity extraction uses heuristic regular expressions; while highly effective on standard naming conventions, unconventional terminology may produce false negatives.
3. Complex multi-column tabular matrices are flattened to linear text during vectorization.

Future enhancements include:
• Bidirectional ARXML AST Parsing: Generating formal AUTOSAR XML schema models directly from HLD documentation.
• Optical Character Recognition (OCR): Incorporating Tesseract for legacy scanned documents.
• Real-Time Bus Timing Simulation: Integrating with CAN/LIN/Ethernet network simulators for communication latency analysis.
• Fine-Tuned Domain NER: Training parameter-efficient domain models for specialized entity recognition.

CHAPTER 10: CONCLUSION & REFERENCES

AUTOSAR AI successfully demonstrates the viability of an on-premises, privacy-preserving document intelligence platform tailored for the automotive software engineering domain. By coupling high-speed PDF ingestion with hybrid dense-plus-BM25 retrieval, local Ollama generative inference, claim decomposition, and architectural rule checking, the platform significantly accelerates architectural audits while eliminating hallucination risks and preserving data privacy.

REFERENCES:
[1] AUTOSAR Consortium, “Explanation of AUTOSAR Adaptive Platform Design,” Technical Specification, Rel. 22-11, 2022.
[2] International Organization for Standardization, “ISO 26262: Road vehicles — Functional safety,” ISO Standard, 2018.
[3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in NeurIPS, vol. 33, pp. 9459–9474, 2020.
[4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in EMNLP, 2019.
[5] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Found. Trends Inf. Retr., vol. 3, no. 4, 2009.
[6] Meta AI, “The Llama 3 Herd of Models,” arXiv preprint arXiv:2407.21783, 2024.
[7] J. M. McKie, “PyMuPDF: High-performance PDF parsing for Python,” Artifex Software, 2024.